

KUBERNETES

What is Kubernetes

- Container orchestration tool
- Automates **deployment, scaling, healing, load balancing**
- It **manages containers automatically** so you don't have to do manual work.
-

Kubernetes does 4 major tasks:

1. **Deploys containers automatically**
2. **Scales containers** (increases/decreases based on traffic)
3. **Self-heals** (restarts crashed containers)
4. **Load balances traffic**

Kubernetes works with **Docker, Containerd**, and other runtimes.

Kubernetes Architecture

Kubernetes has two big parts:

- 1 Control Plane (Brain of the cluster)
- 2 Worker Nodes (Place where your apps run)

Control Plane:

This is the brain or head office of Kubernetes.

It decides, manages, controls, and stores cluster state.

Component	What it does (Easy explanation)
API Server	Entry point for ALL Kubernetes commands (kubectl get pods). It receives requests and passes to other components.
etcd	A key-value database. Stores the entire cluster state (which pods exist, replicas, nodes, etc.)
Controller Manager	Maintains desired state. Example: If you want 3 pods but 1 dies → it creates a new one.
Scheduler	Decides which worker node will run the new pod. Looks at CPU, RAM, node health, taints/tolerations.

- API Server → receives this request
- Scheduler → chooses best node for each pod
- Controller Manager → ensures 3 pods are always running
- etcd → stores this information

Worker Node:

Worker nodes are like servers.

Your containers/pods run here.

Component	What it does (Easy explanation)
kubelet	Talks to API Server. Takes instructions: “Run this pod”, “Delete this pod”. Ensures pod is healthy.
kube-proxy	Manages network rules. Allows pods to talk to each other and services.
Container Runtime	Actually runs the container. Examples: Docker, Containerd, CRI-O

If the Scheduler sends a pod to Node-1:

- kubelet on Node-1 receives the command
- It runs the pod using Docker/Containerd
- kube-proxy sets up networking so the pod can be accessed

Core Objects :

1 Pod:

A Pod is the smallest unit in Kubernetes.

It is basically a wrapper around one or more containers.

Key Points

- Pod = 1 container (mostly used)
- Sometimes Pod = multiple containers (like sidecar logging agent)
- Pods do not self-heal (Deployments and ReplicaSets manage them)

If a container crashes → the pod also dies → Kubernetes recreates a new pod.

2. REPLICASETS:

What ReplicaSet does:

It ensures the number of pods you want is always running.

Example

You want 3 pods, but 1 crashes → ReplicaSet creates a new one to maintain 3/3.

Simple Formula:

Pod dies → ReplicaSet replaces it.

3. DEPLOYMENTS:

A Deployment is a Kubernetes object that manages ReplicaSets and Pods.

Deployment gives you 3 powerful features:

1 Rolling Updates (Zero Downtime)

When you update your app image:

- Kubernetes deletes old pods one by one
- Creates new pods one by one
→ Your application never goes down.

2 Rollbacks (Go back to older version)

If new version fails, misbehaves or crashes:

kubectl rollout undo deployment nginx-deploy

→ Instantly goes back to previous working version.

3 Ensures Desired State

If you say “I want 3 pods”, Deployment ensures:

- always 3 running
- if 1 pod dies → creates a new one

This uses ReplicaSet internally.

Deployment controls ReplicaSet
ReplicaSet controls Pods

4. SERVICES :

A Service in Kubernetes is a stable network endpoint that exposes a group of pods.

Because:

- Pods get new IP every time they restart
- Pods are created/destroyed by Deployments & ReplicaSets

So you cannot rely on pod IP.

A Service provides a permanent IP + DNS name.

Why do we need a Service?

Services solve these problems:

Problem 1 — Pod IP keeps changing

Pods are temporary.

Every time they restart → **IP changes**.

Service gives a **fixed IP**.

Problem 2 — Load balancing pods

If you have:

- 3 backend pods
Service will distribute traffic automatically.
(No external load balancer needed)

Problem 3 — Connect apps inside cluster

Frontend → backend

Backend → database

This communication happens through **Service**.

Problem 4 — Expose app outside cluster

You cannot access pods directly; they are internal.

Service exposes them:

- NodePort → expose on node IP

- LoadBalancer → expose via cloud LB
- Ingress → HTTP/HTTPS routing (works with service)

How Service Works Internally

Services use 2 things:

1 Label Selector

Service finds pods using labels:

Service selects pods:

selector:

app: myapp

Pods have labels:

labels:

app: myapp

Service will auto-detect pods with this label.

2 kube-proxy

Every node runs **kube-proxy**.

It:

- Adds iptables/ipvs rules
- Handles load balancing
- Ensures packets reach correct pod

→ This is how traffic is routed inside cluster.

Types of Services

1. ClusterIP (Default)

✓ Purpose

- Internal communication only
- Not accessible outside cluster

✓ Use Cases

- Backend
- Database
- Internal microservices

✓ How traffic works

Pod → Service → Other Pod

✓ Example:

frontend → backend → database

ClusterIP is used in backend and database layers

2. NodePort

✓ Purpose

Exposes the application to the outside world using:

<Node-IP>:<NodePort>

NodePort range:

30000–32767

✓ How it works

User → NodePort → Service → Pod

✓ Use Cases

- Local testing
- Kubernetes clusters without cloud load balancer
- Access app via node IP

Not used in production

3. LoadBalancer

✓ Purpose

Exposes your app to Internet using Cloud Load Balancer.

Works only on:

- AWS (ELB)
- Azure
- GCP
- DigitalOcean
- Oracle Cloud

✓ Flow:

Internet → Cloud LoadBalancer → NodePort → Pod

✓ Use Cases

- Production workloads
- Public websites
- APIs

✓ Benefits

- Automatic load balancing
- Health checks
- Supports HTTPS (with Ingress)

4. ExternalName

✓ Purpose

Maps service to an external DNS address.

Example:

api.externaldb.com

Kubernetes creates a CNAME record.

✓ Use Case

- External database (RDS, MongoDB Atlas)

- External APIs
- When service is outside Kubernetes cluster

✓ Example:

external-db.default.svc → database.example.com

How Services Load Balance Pods

Service load-balances traffic using:

✓ Round-Robin

Each request goes to the next pod.

✓ IPVS / iptables

kube-proxy uses one of these for routing.

✓ Works automatically

No configuration needed.

5. CONFIGMAPS & SECRETS

These two objects store configuration for your applications

ConfigMap

A ConfigMap is used to store application configuration that is NOT sensitive.

Examples:

- APP_MODE = prod
- DB_HOST = mysql
- PORT = 8080
- LOG_LEVEL = debug

✓ Stores plain text

✓ Good for environment variables

Where ConfigMaps are used?

- To store app settings

- To store URLs
- To store non-secret credentials
- To store entire config files (nginx.conf, app.properties)

Secret :

Secrets are used to store sensitive data that you do NOT want to expose in plain text.

Examples of sensitive data stored in Secrets

- Password
- API Keys
- Tokens
- Certificates
- SSH keys
- Database credentials

Because these values are sensitive, Kubernetes treats them differently than ConfigMaps.

6. VOLUMES

Why do we need Volumes?

Because pods are temporary.

- If a pod is restarted
- If a pod is rescheduled to another node
- If Deployment replaces pods during update

→ All data inside the container is lost.

PV & PVC — Persistent Storage

This is the real solution for permanent storage.

PV = Persistent Volume

Actual storage provided by admin or cloud:

- AWS EBS
- Azure Disk

- GCP Persistent Disk
- NFS
- Ceph
- Local disk

PVC = Persistent Volume Claim

Application's request for storage:

- "I need 10GB"
- "I need ReadWriteOnce access"

PVC automatically binds to matching PV.

7. StatefulSet & StorageClass

A StatefulSet is a Kubernetes controller used to run stateful applications — apps that need stable identity, stable storage, and ordered startup/shutdown.

It is mainly used for databases.

✓ StatefulSet provides:

1. Stable Pod Names

Pods are named with an index:

mysql-0

mysql-1

mysql-2

2. Stable Storage

Each pod gets its own PersistentVolumeClaim (PVC):

mysql-data-mysql-0

mysql-data-mysql-1

3. Ordered Deployment

Pods start one-by-one:

0 → 1 → 2

4. Ordered Termination

Shut down in reverse order:

2 → 1 → 0

StorageClass

A StorageClass defines *how* storage should be created dynamically.

Instead of manually creating PVs → StorageClass automatically creates them when PVC requests storage

✓ StorageClass = Template for storage

It defines:

- Storage type
- Disk type
- Provisioner (AWS, GCP, Azure, local storage)
- Volume binding mode
- Reclaim policy

Why StorageClass Exists?

Without StorageClass → You must manually create PersistentVolumes (PV).

With StorageClass: ✓ PVC automatically creates a PV

✓ No manual intervention

✓ Perfect for dynamic cloud storage

StatefulSet + StorageClass + PVC

When you deploy a StatefulSet:

Step 1 — You define a volumeClaimTemplates section

StatefulSet automatically creates one PVC per pod.

Step 2 — PVC uses the StorageClass

PVC requests storage → StorageClass provisions a disk.

Step 3 — Pod always gets same storage

Even if pod restarts:

- mysql-0 always gets its own disk
- mysql-1 gets its own disk

Full Example Workflow

StatefulSet → creates Pod mysql-0 → creates PVC → StorageClass → creates PV

StatefulSet → creates Pod mysql-1 → creates PVC → StorageClass → creates PV

8. Ingress

Ingress is a Kubernetes object that manages HTTP/HTTPS routing to your services.

Think of Ingress as a smart entry gate that sends incoming traffic to the correct service.

With Ingress:

✓ We expose multiple services through one load balancer

✓ We route traffic based on:

- Domain name
- Subdomain
- URL path

✓ We get HTTPS/SSL support

✓ We get rules-based routing

Ingress Architecture

To use Ingress, we need 2 things:

1 Ingress Resource

This defines:

- which domain
- which URL path
- goes to which service

Example:

/app1 → service app1

/app2 → service app2

2 Ingress Controller

Ingress is only a configuration object — it cannot route traffic by itself.

To route HTTP/HTTPS traffic:

- Manage load balancing
- Terminate SSL (HTTPS)
- Handle URL rules
- Enforce rewrites
- Apply path and host-based rules

→ You MUST install an Ingress Controller.

Common controllers:

- Nginx Ingress Controller (most used)
- AWS ALB Ingress Controller (EKS)
- Istio Gateway

The Ingress Controller watches Ingress rules and configures routing.

Types of Routing Ingress Supports

1 Path-based Routing

/app1 → app1-service

/app2 → app2-service

2 Host-based Routing

api.myapp.com → backend-service

www.myapp.com → frontend-service

3 TLS/HTTPS Routing

Ingress handles SSL certificates using:

- cert-manager
- Let's Encrypt

Ingress Advantages

- ✓ One single entry point
- ✓ Multiple apps exposed with one load balancer
- ✓ Supports domain/subdomain routing
- ✓ Supports path routing
- ✓ Free HTTPS using cert-manager
- ✓ Clean URL (no NodePort numbers!)

9. Namespaces

Why important?

- Divide cluster into logical sections
- Separate Dev / QA / Prod environments
- Limit access with RBAC

Namespace = logical isolation inside cluster.

Used to separate teams, envs, resources.

Default namespaces: default, kube-system, kube-public, kube-node-lease

10 . Labels,Selectors & Annotations

Labels = key-value metadata used for selecting pods.

Selectors = used by services and deployments to match pods.

Annotations = extra non-identifying metadata.

Yaml for every Kubernetes object

THE 4 BASIC SECTIONS INSIDE ANY KUBERNETES YAML

apiVersion: kind: metadata: spec:

1. apiVersion: (Which API group this object belongs to)

Example:

YAML

apiVersion: apps/v1

✓ What it means:

This tells Kubernetes which version of the API this object uses.

✓ Why this exists:

Kubernetes has many API groups, like:

- v1
- apps/v1
- rbac.authorization.k8s.io/v1
- networking.k8s.io/v1

Different resources use different API groups.

✓ Example API versions:

Resource	apiVersion
Pod	v1
Service	v1
Deployment	apps/v1
ReplicaSet	apps/v1
StatefulSet	apps/v1
Ingress	networking.k8s.io/v1
ConfigMap	v1
Secret	v1

If apiVersion is wrong → YAML will fail.

2. kind: (What type of resource you are creating)

Example:

YAML

kind: Deployment

✓ Meaning:

This tells Kubernetes **which object** this YAML creates.

✓ Common kinds you use:

Kind	Meaning
Pod	Run a single pod
Deployment	Manage Pods + updates
ReplicaSet	Maintain pod count
Service	Expose pods
ConfigMap	Store config
Secret	Store sensitive data
PersistentVolumeClaim	Request storage
StatefulSet	Stateful apps
Ingress	HTTP routing

3. metadata: (Basic info: name, labels, namespace)

Example:

YAML

metadata:

name: myapp

labels:

app: myapp

✓ Contains:

- name: → name of object
- labels: → used by services, deployments, selectors
- namespace: (optional)

✓ Why labels matter?

Services use labels to find pods:

Service:

YAML

selector:

app: myapp

Pod:

YAML

labels:

app: myapp

This is how **Service connects to Pods**.

4. spec: (The main configuration — the brain)

This is the **most important** part of YAML.

Everything inside spec: defines the real behaviour of the resource.

But **spec changes depending on the kind**.

Let's break it **for each object**.

HOW spec WORKS FOR EACH OBJECT

◆ Pod spec

YAML

spec:

containers:

Contains:

- containers
- image
- ports
- env
- volumes

◆ **Deployment spec**

YAML

spec:

replicas:

selector:

template:

Inside template: is another spec:
(Template is basically Pod YAML)

◆ **ReplicaSet spec**

YAML

spec:

replicas:

selector:

template:

Same pattern as Deployment, but without rolling update features.

◆ **Service spec**

YAML

spec:

selector:

ports:

type:

- selector → selects pod
- ports → exposes ports
- type → ClusterIP / NodePort / LoadBalancer

◆ **ConfigMap spec**

Actually ConfigMap uses data: instead of spec:.

YAML

data:

key: value

◆ **Secret spec**

Also uses data: but base64 encoded.

◆ **PVC spec**

YAML

spec:

accessModes:

resources:

requests:

storage:

◆ **StatefulSet spec**

YAML

spec:

serviceName:

replicas:

selector:

template:

volumeClaimTemplates:

◆ Ingress spec

YAML

spec:

rules:

- host:

http:

paths:

Section	Meaning
apiVersion	Which API group does the object belong to
kind	What resource are you creating
metadata	Name + labels
spec	Actual configuration (varies by resource)

Everything else is **inside spec**, depending on resource type.

1. POD YAML

apiVersion: v1

kind: Pod

metadata:

name: mypod

labels:

```
  app: myapp
spec:
  containers:
  - name: nginx
    image: nginx:latest
    ports:
    - containerPort: 80
```

2. REPLICASET YAML

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: myapp-rs
spec:
  replicas: 3
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
      - name: nginx
        image: nginx:latest
```

3.DEPLOYMENT YAML

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-deploy
spec:
  replicas: 3
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: nginx
          image: nginx:1.21
          ports:
            - containerPort: 80
```

4. SERVICES YAML

ClusterIP (Default)

```
apiVersion: v1
kind: Service
metadata:
  name: myapp-clusterip
spec:
```

selector:

app: myapp

ports:

- port: 80

targetPort: 80

NodePort

apiVersion: v1

kind: Service

metadata:

name: myapp-nodeport

spec:

type: NodePort

selector:

app: myapp

ports:

- port: 80

targetPort: 80

nodePort: 30080

LoadBalancer

apiVersion: v1

kind: Service

metadata:

name: myapp-lb

spec:

type: LoadBalancer

selector:

```
  app: myapp
  ports:
  - port: 80
  targetPort: 80
```

ExternalName

```
apiVersion: v1
kind: Service
metadata:
  name: external-db
spec:
  type: ExternalName
  externalName: database.example.com
```

5. CONFIGMAP YAML

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_MODE: "prod"
  DB_HOST: "mysql"
  PORT: "8080"
```

6. SECRET YAML

```
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
```


type: Opaque

data:

password: bXlwYXNzd29yZA==

7. PVC (Persistent Volume Claim) YAML

apiVersion: v1

kind: PersistentVolumeClaim

metadata:

name: app-pvc

spec:

accessModes:

- ReadWriteOnce

resources:

requests:

storage: 5Gi

8. STORAGECLASS YAML

apiVersion: storage.k8s.io/v1

kind: StorageClass

metadata:

name: standard

provisioner: kubernetes.io/aws-ebs

parameters:

type: gp2

reclaimPolicy: Retain

volumeBindingMode: WaitForFirstConsumer

9. STATEFULSET YAML

apiVersion: apps/v1

kind: StatefulSet

metadata:

name: mysql

spec:

serviceName: mysql

replicas: 3

selector:

matchLabels:

app: mysql

template:

metadata:

labels:

app: mysql

spec:

containers:

- name: mysql

image: mysql:8

volumeMounts:

- name: data

mountPath: /var/lib/mysql

volumeClaimTemplates:

- metadata:

name: data

spec:

accessModes: ["ReadWriteOnce"]

storageClassName: standard

resources:

requests:

storage: 10Gi

10. INGRESS (Path-Based Routing) YAML

apiVersion: networking.k8s.io/v1

kind: Ingress

metadata:

name: my-ingress

annotations:

nginx.ingress.kubernetes.io/rewrite-target: /

spec:

rules:

- http:

paths:

- path: /app1

pathType: Prefix

backend:

service:

name: app1-svc

port:

number: 80

- path: /app2

pathType: Prefix

backend:

service:

name: app2-svc

port:

number: 80

SCALING & AUTOHEALING

Kubernetes automatically heals and scales applications using:

✓ **HPA (Horizontal Pod Autoscaler)**

✓ **VPA (Vertical Pod Autoscaler)**

✓ **Cluster Autoscaler**

Horizontal Pod Autoscaler (HPA)

Scales number of PODS based on CPU, Memory, or Custom Metrics.

Example:

- CPU > 80% → scale from 3 pods → 6 pods
- CPU < 20% → scale down from 6 → 3 pods

YAML Example:

apiVersion: autoscaling/v2

kind: HorizontalPodAutoscaler

metadata:

name: myapp-hpa

spec:

minReplicas: 2

maxReplicas: 10

metrics:

- type: Resource

resource:

name: cpu

target:

type: Utilization

averageUtilization: 70

scaleTargetRef:

apiVersion: apps/v1

kind: Deployment

name: myapp

HPA uses:

- CPU metrics
- Memory metrics
- Prometheus metrics (custom)

Vertical Pod Autoscaler (VPA)

Adjusts CPU and memory of POD itself.

HPA = Scale OUT (more replicas)

VPA = Scale UP (bigger pod resources)

Example:

- Pod struggling? VPA increases CPU/RAM
- Pod over-provisioned? VPA reduces CPU/RAM

Mostly used with batch jobs, ML workloads

Not used with HPA at the same time (avoid conflict)

Cluster Autoscaler

Adds or removes NODES automatically.

HPA scales pods →

Cluster Autoscaler ensures nodes are available.

How it works:

- If pods cannot be scheduled →
Add new nodes
- If nodes are underutilized →
Remove nodes

Used in cloud:

- AWS EKS
- Azure AKS
- GKE

SCALING SUMMARY TABLE

Autoscaler	What it Scales	Example
HPA	Pods	3 pods → 10 pods
VPA	CPU/RAM of pod	200m CPU → 400m
Cluster Autoscaler	Nodes	3 nodes → 5 nodes

Kubernetes auto-heals using:

Feature	What it fixes
Liveness Probe	Dead app → restart
Readiness Probe	Not ready app → remove from LB
Startup Probe	Slow app startup → restart
ReplicaSet	Pod died → new pod
Deployment	Crash during rollout → rollback
HPA	High load → more pods
Cluster Autoscaler	No space → add nodes

Probes

Probes are health checks to monitor containers inside Pods.

Kubernetes uses 3 types of probes:

- 1 Liveness Probe – Is the app alive?
- 2 Readiness Probe – Is the app ready to handle traffic?
- 3 Startup Probe – Did the app start correctly?

Each probe decides what action Kubernetes should take.

LIVENESS PROBE —

“Is the app ALIVE?”

✓ Purpose

To check whether your application is healthy and running without being stuck.

If liveness probe fails, Kubernetes will:

Kill the container and restart it automatically.

Why needed?

Some apps:

- hang
 - go into infinite loop
 - become unresponsive
- but the process is still running — so Kubernetes can't detect failure without a probe.

Real Example:

Your app stops responding at /health.

Liveness Probe detects:

/health → no response

Action:

Kubernetes restarts the container.

Liveness YAML

livenessProbe:

httpGet:

path: /health

port: 8080

initialDelaySeconds: 10

periodSeconds: 5

READINESS PROBE

“Can the app receive traffic NOW?”

✓ Purpose

Checks if the application is ready to serve requests.

Even if the container is RUNNING, it may not be ready (ex: DB not connected).

If readiness probe fails, Kubernetes will:

Remove the Pod from the Service load balancer.

No traffic will be sent to this pod.

Real Example:

Pod is running, but waiting for:

- DB connection
- Config load
- Cache warm-up

Readiness probe fails → user traffic stops going to that pod.

Readiness YAML

readinessProbe:

httpGet:

path: /ready

port: 8080

initialDelaySeconds: 5

periodSeconds: 5

STARTUP PROBE

“Has the app finished STARTING?”

✓ Purpose

Checks if a slow-starting application has successfully started.

Used for:

- Java Spring Boot
- Node.js heavy apps
- Python apps loading big models

If startup probe fails, Kubernetes will:

Restart the container

Liveness and Readiness probes will NOT run until Startup probe passes

Startup YAML

startupProbe:

httpGet:

path: /startup

port: 8080

failureThreshold: 30

periodSeconds: 10

This gives the app 300 seconds (30×10) to start.

How All 3 Probes Work Together

Pod starting phase

Only Startup Probe runs.

If startup succeeds → liveness + readiness start

If startup fails → container restarted

After app has started

Probe	Purpose	Action on Fail
Liveness	App healthy?	Restart container

Readiness	App ready for traffic?	Remove from serv
------------------	------------------------	------------------

RBAC in Kubernetes

RBAC = Role-Based Access Control

It controls WHO (user/service account) can do WHAT (permissions) on WHICH resources in Kubernetes.

RBAC decides who can access what in the cluster

Why RBAC is Needed?

Without RBAC:

- Anyone could delete pods
- Anyone could exec into containers
- Anyone could modify deployments
- Anyone could see secrets

With RBAC: ✓ Give limited access

✓ Protect secrets

✓ Prevent mistakes

✓ Control developer access

✓ Secure production clusters

RBAC Involves 4 Components

Component	Meaning
Role	Set of permissions within a namespace
ClusterRole	Permissions at cluster-wide level
RoleBinding	Attach Role → user/service account
ClusterRoleBinding	Attach ClusterRole → user/service account

Component	Meaning
-----------	---------

Role	WHAT can be done (namespace-level)
ClusterRole	WHAT can be done (cluster-level)
RoleBinding	WHO gets Role
ClusterRoleBinding	WHO gets ClusterRole

1. Cluster Commands

kubectl version # Show client & server Kubernetes version

kubectl cluster-info # Show cluster master and DNS details

kubectl get nodes # List all worker/master nodes

kubectl describe node <node> # Detailed info of a node

kubectl top node # Show CPU/memory usage of each node

2. Get Commands

kubectl get pods # List pods in current namespace

kubectl get pods -o wide # Pods with node, IP, container info

kubectl get pods -A # List pods in ALL namespaces

kubectl get svc # List all services

kubectl get deployments # List deployments

kubectl get rs # List ReplicaSets

kubectl get cm # List ConfigMaps

kubectl get secret # List secrets

kubectl get pvc # List PersistentVolumeClaims

kubectl get pv # List PersistentVolumes

kubectl get sc # List StorageClasses

kubectl get ns # List all namespaces

kubectl get ingress # List ingress rules

3. Describe Commands

kubectl describe pod <pod> # Detailed info of a pod

kubectl describe deployment <name> # Deployment details

kubectl describe svc <svc> # Service details

kubectl describe node <node> # Node details

kubectl describe pvc <pvc> # PVC details

kubectl describe pv <pv> # PV details

kubectl describe ingress <name> # Ingress details

4. Create Commands

kubectl create namespace dev # Create namespace

kubectl create deployment nginx --image=nginx # Create deployment

kubectl create configmap app-config --from-literal=key=value # Create ConfigMap

kubectl create secret generic db-secret --from-literal=password=admin123 # Create Secret

5. Apply & Delete

kubectl apply -f file.yaml # Apply YAML (create/update)

kubectl delete -f file.yaml # Delete resources from YAML

kubectl delete pod <pod> # Delete a pod

kubectl delete deployment <name> # Delete a deployment

kubectl delete svc <svc> # Delete service

kubectl delete ns <namespace> # Delete namespace

6. POD Commands

kubectl get pods # List pods

kubectl logs <pod> # Show pod logs

kubectl logs <pod> -f # Stream logs (follow)

kubectl exec -it <pod> -- bash # Enter pod's shell (bash)

kubectl exec -it <pod> -- sh # Enter shell (sh)

kubectl port-forward pod/<pod> 8080:80 # Forward pod port to local

7. Deployment Commands

kubectl get deployments # List deployments

kubectl describe deployment <name> # Deployment details

kubectl scale deployment <name> --replicas=5 # Scale deployment

kubectl rollout status deployment <name> # Check rollout status

kubectl rollout undo deployment <name> # Rollback to previous version

kubectl set image deployment/myapp myapp=nginx:1.25 # Update container image

8. Service Commands

kubectl get svc # List services

kubectl describe svc <name> # Describe service

kubectl expose deployment myapp --port=80 --type=NodePort # Expose deployment

kubectl expose pod nginx --type=ClusterIP --port=80 # Expose pod internally

9. Namespace Commands

kubectl get ns # List namespaces

kubectl create ns dev # Create namespace

kubectl delete ns dev # Delete namespace

kubectl get pods -n kube-system # Pods in kube-system namespace

kubectl get svc -n dev # Services in dev namespace

10. ConfigMap Commands

kubectl get cm # List ConfigMaps

kubectl describe cm <name> # Show ConfigMap details

kubectl create configmap game-config --from-literal=mode=hard # Create CM (literal)

kubectl create configmap app-cm --from-file=app.properties # Create CM from file

11. Secret Commands

kubectl get secret # List secrets

kubectl describe secret <name> # Describe secret

kubectl create secret generic mysecret --from-literal=password=pass123 # Create secret

12. PVC & PV Commands

kubectl get pvc # List PVCs

kubectl get pv # List PVs

kubectl describe pvc <name> # PVC details

kubectl describe pv <name> # PV details

13. Ingress Commands

kubectl get ingress # List ingress

kubectl describe ingress <name> # Show routing rules

14. RBAC Commands

kubectl get roles -n dev # List Roles in namespace

kubectl get rolebindings -n dev # RoleBindings in namespace

kubectl get clusterroles # List all ClusterRoles

kubectl get clusterrolebindings # List ClusterRoleBindings

15. Events & Troubleshooting

kubectl get events # Show cluster events

kubectl describe pod <pod> # Show pod issues

kubectl logs <pod> # Check logs

kubectl exec -it <pod> -- bash # Debug inside pod

16. HPA (Horizontal Pod Autoscaler)

kubectl get hpa # List HPA

kubectl describe hpa <name> # HPA details

kubectl autoscale deployment myapp --min=2 --max=10 --cpu-percent=70 # Create HPA

17. Node Maintenance

kubectl cordon <node> # Mark node unschedulable

kubectl uncordon <node> # Make node schedulable

kubectl drain <node> --ignore-daemonsets --force # Evict pods and drain node

18. Edit Resources

kubectl edit deployment <name> # Edit deployment

kubectl edit svc <name> # Edit service

kubectl edit cm <name> # Edit ConfigMap

19. Generate YAML (Dry Run)

kubectl create deployment nginx --image=nginx -o yaml --dry-run=client # Generate YAML

kubectl apply -f deploy.yaml --dry-run=client # Validate YAML

20. Useful Shortcuts

alias k=kubectl # Create alias

k get pod # Shortcut for kubectl get pods

k get all # Show all resources

k logs -f <pod> # Follow logs